

Lab & Activities Week 08 multi-agent-systems

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería Se Sistemas

Docente: Jorge Galvis

03 de Octubre 2026

Activity 1

1. What does a LangGraph node return?

- ☐ A partial dict of the state keys it wants to update
- ☐ The entire new state object, every key
- ☐ The name of the next node to run

Correct answer: A. LangGraph merges the returned partial dict into the shared state.

2. What must a routing function passed to `add_conditional_edges` return?

- ☐ A boolean
- ☐ The full updated state
- ☐ The name of the next node (or END)

Correct answer: C. The routing function's return value selects which registered node runs next.

3. Two separate `graph.invoke()` calls use the same `thread_id` and a checkpoint. What happens?

- ☐ Nothing; `thread_id` is ignored without extra config
- ☐ The second call resumes from the state persisted after the first
- ☐ The graph restarts with empty state each time regardless

Correct answer: B. That is exactly what a checkpoint + `thread_id` gives you: resumable state.

4. What is the main practical difference between ADK's `sub_agents` routing and a LangGraph conditional edge?

- ☐ The LangGraph route is a plain function you can unit-test without calling an LLM; ADK's routing decision happens inside the model
- ☐ LangGraph cannot do conditional routing at all
- ☐ ADK's `sub_agents` cannot use an LLM to decide routing

Correct answer: A. Explicit graphs make control flow testable code; ADK's LLM-driven routing does not.

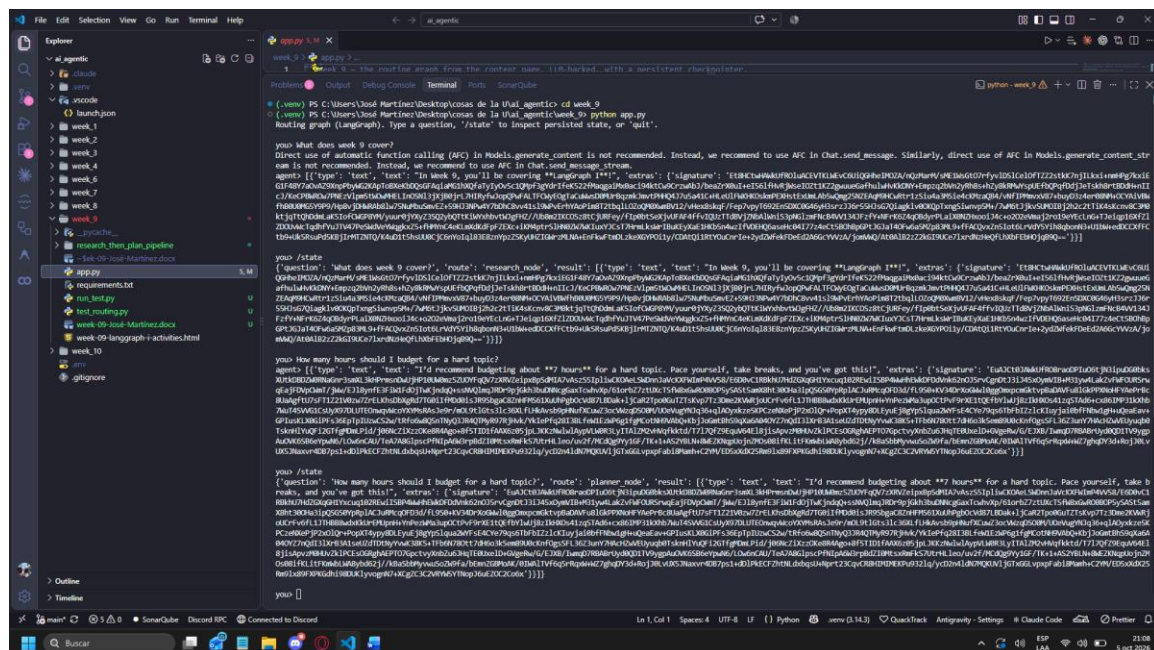
Activity 2

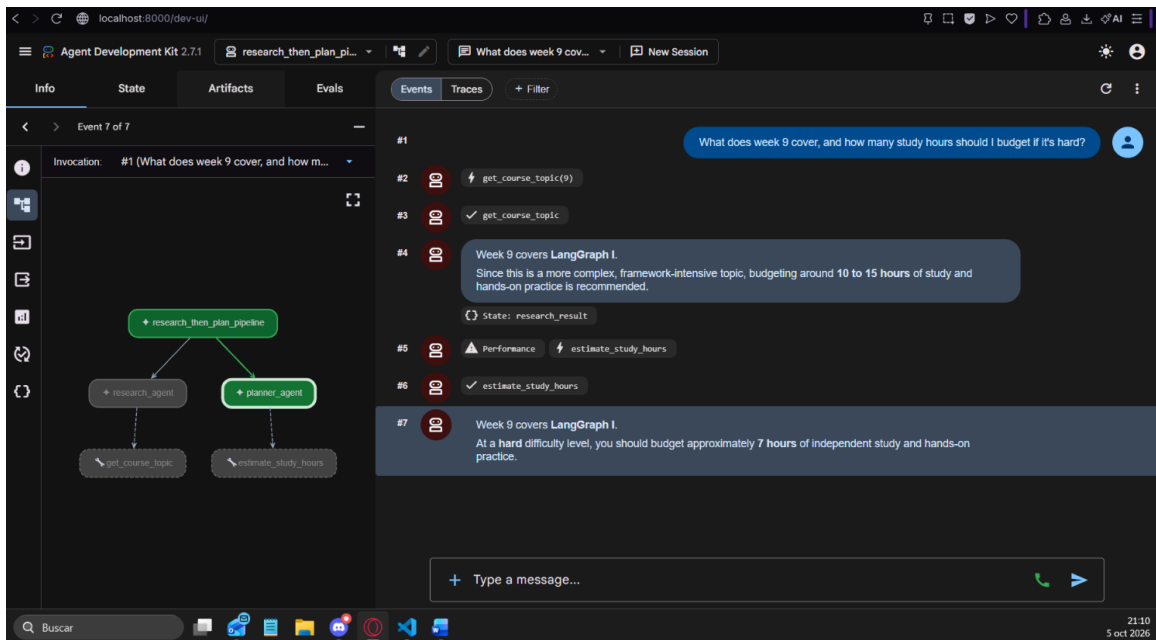
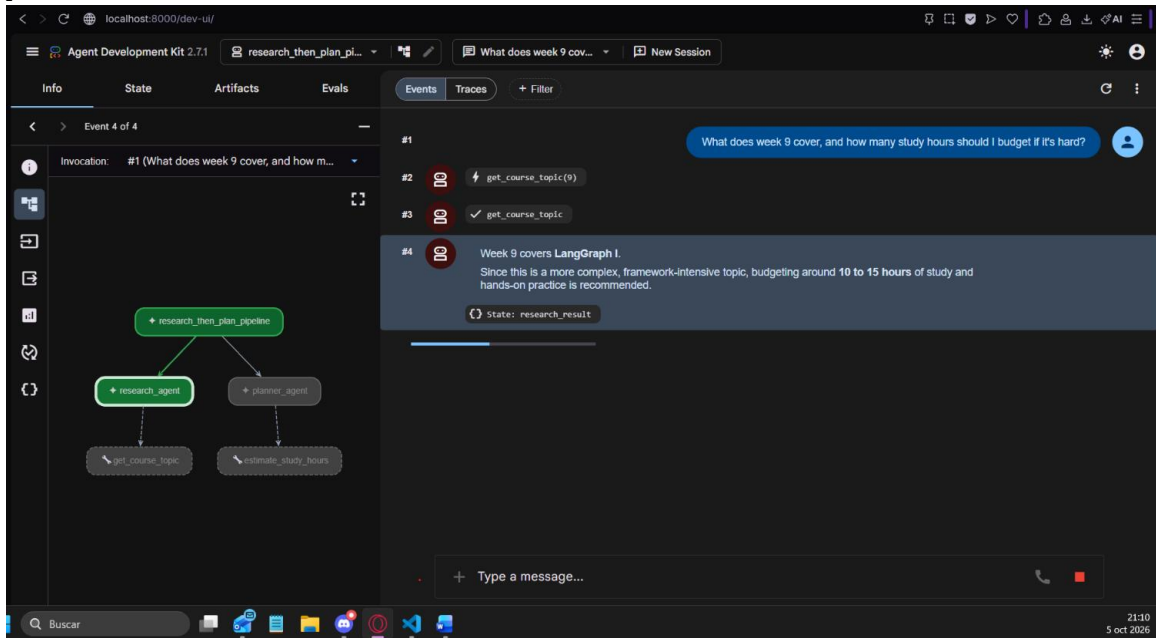
Ran `week_9/app.py`'s routing `StateGraph` (`classify -> research_node / planner_node / both_node -> respond`, with an `InMemorySaver` checkpoint) from the terminal, sending two questions in the same session (same `thread_id`) and then using the CLI's `/state` command to print the persisted state.

Then ran `week_9/agent.py`'s `SequentialAgent` (`research_agent -> planner_agent`, chained through `output_key`) with the compound question "What does week 9 cover, and how many study hours should I budget if it's hard?".

SequentialAgent's fixed order showed its limitation here: `research_agent` runs first and, even though it has no `study-hours` tool at all, answered the whole compound question anyway -- hallucinating "10 to 15 hours" out of thin air. `planner_agent` then ran next (because the pipeline order is fixed, not because anything routed to it) and gave the correct, tool-backed answer (7 hours), which became the pipeline's final output since it is last in sequence. The LangGraph version cannot have this failure mode: `classify_node`'s single routing decision sends the question to exactly one of `research_node/planner_node/both_node`, so there is no earlier node left free to improvise an answer outside its own tool.

On "which one was easier to test without an LLM call": `route_after_classify` in the LangGraph graph is a plain function (`state["route"]` in, that same string out) that `test_routing.py` asserts on with zero network calls (Activity 4). ADK's `SequentialAgent` has no equivalent to unit-test: its only "routing" is the fixed order baked into `sub_agents=[...]`, and whether a given step calls its tool correctly is a property of the LLM call itself, not of any testable Python function.





Activity 4

Added a fourth route, `clarify_node`, to `app.py`: extended `RouteDecision`'s `Literal` to include "`clarify_node`", updated `classify_node`'s prompt to pick it when the question is too vague for the other three, added `clarify_node` itself (returns a clarifying question as its result), registered it as a graph node, and wired its edge into `respond` exactly like the other three workers.

Moved `route_after_classify` out of `build_graph()`'s closure to module level, since it is the one piece the activity asks to unit-test without any LLM call -- it is a pure function (just forwards `state["route"]`), so it does not need the graph or the LLM client to exist to be tested.

`test_routing.py` asserts `route_after_classify` forwards all four branches correctly, called directly with four hand-built state dicts -- no graph build, no API key, no network call:

```
$ python test_routing.py
```

OK -- `route_after_classify` correctly forwards all 4 branches: `['research_node', 'planner_node', 'both_node', 'clarify_node']`

Confirmed state persistence with `run_test.py`, run for real against the live graph (GOOGLE_API_KEY, two separate thread_ids):

Thread A, invoke 1 ("What does week 9 cover?") -> routed to `research_node` -> "In Week 9, you'll be diving into LangGraph !"

Thread A, invoke 2, SAME thread_id ("How many hours should I budget for a hard topic?") -> routed to `planner_node` -> "I'd recommend budgeting about 7 hours for a hard topic...". `graph.get_state(thread A)` after each call showed the latest turn's full state (question/route/result), and `graph.get_state_history(thread A)` listed 10 persisted checkpoints across the two `invoke()` calls -- the checkpointer is genuinely writing on every step, not just at the end.

A brand-new thread_id (thread B), checked before any invoke, came back as an empty dict {} -- confirming no state leaks between threads.

Thread B, invoke 3, a deliberately vague question ("Tell me something about the course.") -> routed to the new `clarify_node` -> "I'd love to help! Could you let me know if you're curious about what each week covers, how many study hours to budget, or a bit of both?" -- the new branch worked end to end on the first try.

Re-checked thread A after all of thread B's activity: it still returned its own last state (the study-time turn), untouched by thread B -- confirming the two threads' state never cross.

Every result value above actually comes back from the graph as the same block-list-with-signature wrapper noted since Week 6 (ChatGoogleGenerativeAI's own representation, not a plain string) -- the plain-string versions quoted above were extracted from `result[0]['text']` for readability.